Meet Flutter

Everything you just learned is already inside every Flutter app

18

FLUTTER TEASER · 1 OF 3

A Widget Is Just a Class

Dart

```
// Widgets are objects
const Text('Hello');
const Icon(Icons.star);

// Widgets contain other widgets
Row(
  children: [
    Icon(Icons.star),
    Text('4.9'),
  ],
)
```

Widget = Dart class
you create it with a constructor

Nest widgets
to build a whole screen

Named parameters
you already know this syntax

19

FLUTTER TEASER · 2 OF 3

Create & Run the Counter App

Terminal

```
$ flutter create counter_app
$ cd counter_app
$ flutter run

counter_app/
  lib/main.dart     <- YOUR code lives here
  pubspec.yaml      <- packages
  android/  ios/    <- generated shells
  test/             <- tests
```

3 commands
create, enter, run

lib/main.dart
the file you will edit

Press r
hot reload: see changes instantly

FLUTTER TEASER · 3 OF 3

Spot the OOP in main.dart

Dart

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(child: Text('Hi')),
      ),
    );
  }
}
```

extends
inheritance, recall last lecture

const + super.key
const constructors & super

@override build()
override one method

SELF-CHECK

Dart Mastery Checklist: Can You...?

01
Read types & null safety
var, final, const, ?, ??

02
Write functions
named and optional parameters

03
Use collections
List, Map, Set and their methods

04
Design a class tree
constructors, extends, @override

05
Share code
mixins, abstract classes, extensions

06
Spot OOP in Flutter
extends, const, build() override

22

COMING UP

Today's Session: Breaking into Flutter

1 **Anatomy of the counter app**
main(), runApp, the widget tree

2 **Stateless vs Stateful**
setState and the widget lifecycle

3 **BuildContext & composition**
where a widget lives, building small

4 **Layout + hands-on**
Row, Column, then enhance the counter

By the end of this class: (1) Make sure flutter doctor is clean (2) Run flutter create counter_app (3) Perform the exercise that will be given

23

CS 442 | Mobile Application Development

Breaking into Flutter

LECTURE — ~60 MIN

Widgets, state, BuildContext & layout basics

HANDS-ON — ~20-30 MIN

Enhance the counter app yourself

Based on Flutter in Action, Ch. 3 (Breaking into Flutter)

24

TODAY'S SESSION

Today's Session — Roadmap

LECTURE — ~60 MIN

1. **Anatomy of the counter app** ~9 min
2. **build() & hot reload** ~7 min
3. **Stateless, Stateful & setState** ~10 min
4. **Lifecycle & BuildContext** ~10 min
5. **Composition over inheritance** ~5 min
6. **Layout basics & widget toolbox** ~17 min

HANDS-ON — ~28 MIN

1. **Run it & hot reload** ~3 min
2. **Add a Reset button** ~5 min
3. **Tinker with the Timer** ~5 min
4. **Run the Timer without and with in Counter** ~10 min
5. **Bonus: no negative counts** ~5 min

25

WARM-UP · 3 MIN

Quick Check From Last Session

1. **What does @override do?** Hint: Dog.speak() versus Animal.speak()
2. **extends or implements?** Hint: do you want the parent's code, or only its shape?
3. **Why write const widgets?** Hint: could Flutter skip rebuilding it?

ANATOMY OF THE COUNTER APP

How an App Starts

1. `main()` — Every Dart program starts here
2. `runApp()` — Puts your root widget on screen
3. `MyApp` — Stateless: sets up the whole app
4. `MyHomePage` — Stateful: holds the counter

Each step hands over to the next. Open lib/main.dart and find all four.

ANATOMY OF THE COUNTER APP

main.dart: The Skeleton

Dart

```dart
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: MyHomePage(),
    );
  }
}
```

main()
Dart starts here, calls runApp

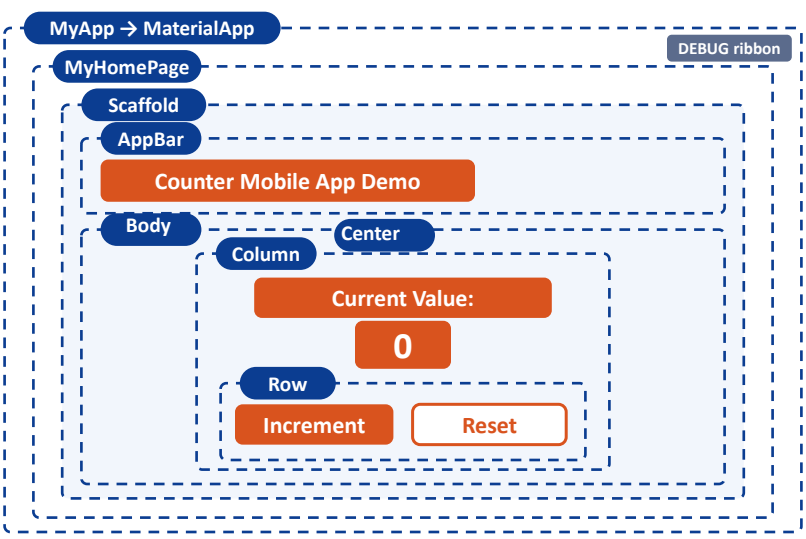
MaterialApp
the app's outer shell

home:
the first screen to show

28

ANATOMY OF THE COUNTER APP

Where Each Widget Sits on the Screen



How to read it

Visible on screen

Invisible: only arranges

Think Java Swing

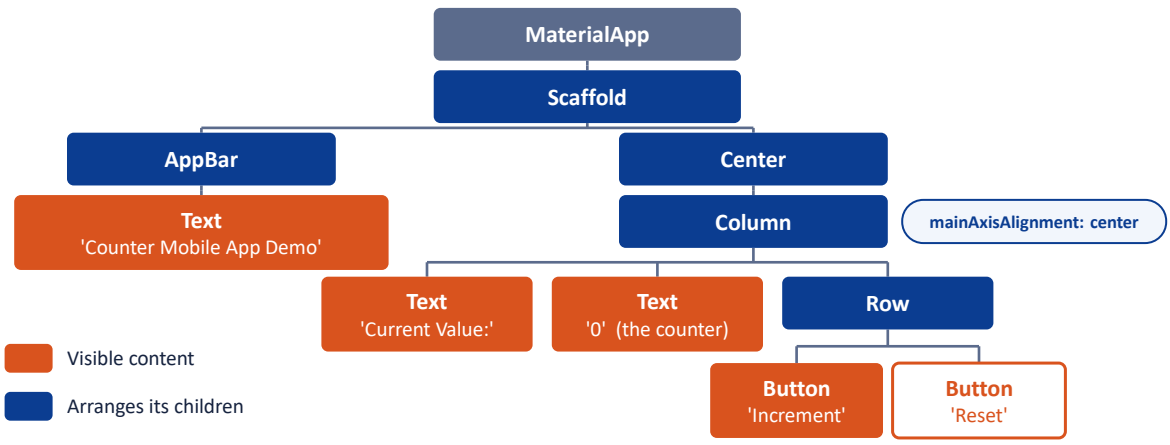
MaterialApp ≈ JFrame
Scaffold ≈ content pane
Row, Column ≈ JPanel
Text ≈ JLabel
Button ≈ JButton

DEBUG ribbon: added by MaterialApp in debug mode

29

EVERYTHING IS A WIDGET

The Widget Tree of the Counter Screen



Widgets nest inside widgets. The whole screen is one tree.

30

THE build() METHOD

build() Describes the Screen

Dart

```dart
@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text('You have pushed:'),
      Text('$_counter'),
    ],
  );
}
```

Returns widgets
a description of the screen

Runs again
whenever the data changes

Keep it quick
no heavy work inside build()

31

DEVELOPER SUPERPOWERS

const Widgets & Hot Reload

C	**const widgets**	A const widget never changes, so Flutter can reuse it
r	**Hot reload (r)**	New code in under a second, and your state is kept
R	**Hot restart (R)**	Restarts the app and resets state; use after editing main()

STATELESSWIDGET

StatelessWidget: Fixed Output

Dart

```dart
class Greeting extends StatelessWidget {
  final String name;
  const Greeting(this.name, {super.key});

  @override
  Widget build(BuildContext context) {
    return Text('Hello, $name!');
  }
}
// Use it:  const Greeting('Ada')
```

No changing data
same input, same output

Data comes in
through final fields

Use for
labels, icons, fixed layouts

STATEFULWIDGET

StatefulWidget: Two Classes

Dart

```dart
class Counter extends StatefulWidget {
  const Counter({super.key});
  @override
  State<Counter> createState() =>
      _CounterState();
}
class _CounterState extends State<Counter> {
  int _count = 0;   // state lives here
  @override
  Widget build(BuildContext context) =>
      Text('$_count');
}
```

Two classes
the widget and its State

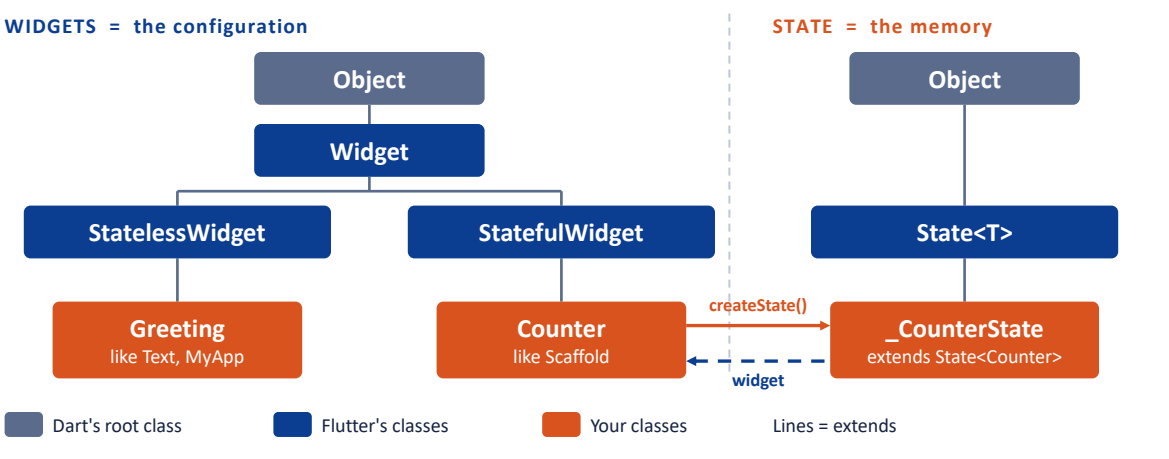
Data lives in State
like int _count

Widget rebuilt
but State survives

34

THE BIG PICTURE

The Widget Family Tree

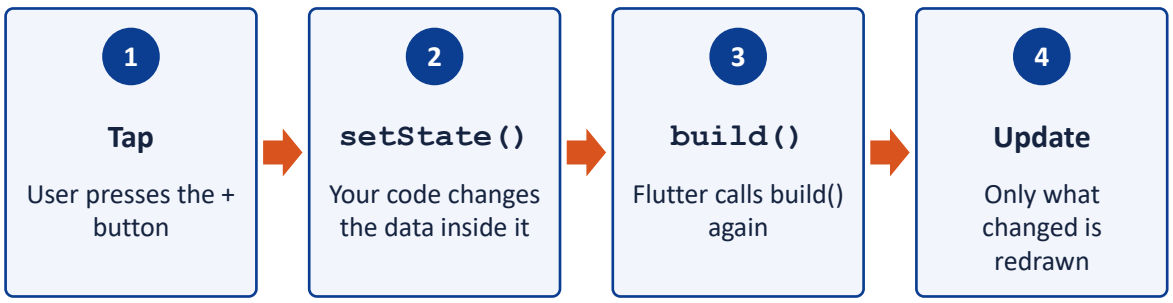


The widget is the configuration. The State is the memory.

35

setState

How the Screen Updates



Changing a variable alone does nothing. setState tells Flutter to rebuild.

36

setState

The Classic Beginner Bug

Without setState

```
onPressed: () {
  _count++;
}
```

Variable changes, screen doesn't

With setState

```
onPressed: () {
  setState(() {
    _count++;
  });
}
```

build() runs again: screen updates

Rule: change data INSIDE setState

37

STATE LIFECYCLE

The Life of a State Object



Call super.initState() first and super.dispose() last

38